

Sobrecarga de constructores C#

VAMOS A GENERAR UN EJEMPLO DE SOBRECARGA EN C#
(dos constructores sobrecargados) para una clase
hipotética (Alumno)

Un ejemplo clásico para explicar **sobrecarga de constructores** es una clase `Alumno` donde un constructor crea el objeto con datos mínimos y otro permite inicializar todos los atributos.

Clase Alumno

```
</> C#  
  
using System;  
  
namespace EjemploSobrecarga  
{  
    public class Alumno  
    {  
        // Atributos  
        private string nombre;  
        private string apellido;  
        private int legajo;  
        private double notaFinal;  
  
        // Constructor 1 (datos mínimos)  
        public Alumno(string nombre, string apellido)  
        {  
            this.nombre = nombre;  
            this.apellido = apellido;  
            this.legajo = 0;  
            this.notaFinal = 0;  
        }  
  
        // Constructor 2 (todos los datos)  
        public Alumno(string nombre, string apellido, int legajo, double notaFinal)  
        {  
            this.nombre = nombre;  
            this.apellido = apellido;  
            this.legajo = legajo;  
            this.notaFinal = notaFinal;  
        }  
  
        // Método para mostrar la información  
        public void MostrarDatos()  
        {  
            Console.WriteLine("Nombre: " + nombre);  
            Console.WriteLine("Apellido: " + apellido);  
            Console.WriteLine("Legajo: " + legajo);  
            Console.WriteLine("Nota Final: " + notaFinal);  
            Console.WriteLine("-----");  
        }  
    }  
}
```

Programa Principal

```
</> C#  
  
using System;  
  
namespace EjemploSobrecarga  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            // Utiliza el primer constructor  
            Alumno alumno1 = new Alumno("Juan", "Pérez");  
  
            // Utiliza el segundo constructor  
            Alumno alumno2 = new Alumno("Ana", "Gómez", 1254, 8.75);  
  
            alumno1.MostrarDatos();  
            alumno2.MostrarDatos();  
  
            Console.ReadKey();  
        }  
    }  
}
```

Salida esperada

```
Nombre: Juan  
Apellido: Pérez  
Legajo: 0  
Nota Final: 0  
-----  
Nombre: Ana  
Apellido: Gómez  
Legajo: 1254  
Nota Final: 8.75  
-----
```

Explicación

En este ejemplo la clase **Alumno** posee **dos constructores** con el mismo nombre (**Alumno**), pero con **diferente cantidad de parámetros**.

```
</> C#  
  
public Alumno(string nombre, string apellido)
```

Este constructor permite crear un alumno indicando únicamente el nombre y el apellido. Los demás atributos toman valores por defecto.

Mientras que:

```
</> C#
```

```
public Alumno(string nombre, string apellido, int legajo, double notaFinal)
```

permite crear un alumno proporcionando toda la información desde el momento de la creación del objeto.

La elección del constructor la realiza automáticamente el compilador según la cantidad y el tipo de parámetros utilizados al instanciar el objeto.

Por ejemplo:

```
</> C#
```

```
Alumno a1 = new Alumno("Juan", "Pérez");
```

invoca el primer constructor.

En cambio:

```
</> C#
```

```
Alumno a2 = new Alumno("Ana", "Gómez", 1254, 8.75);
```

invoca el segundo.

Concepto importante

La **sobrecarga** consiste en definir varios métodos o constructores con el **mismo nombre**, pero con **distinta lista de parámetros** (cantidad, tipo u orden). Esto permite ofrecer diferentes formas de crear o utilizar un objeto sin cambiar el nombre del constructor.

En este caso, ambos constructores se llaman `Alumno`, pero el compilador puede diferenciarlos porque sus firmas son distintas. Este mecanismo mejora la flexibilidad y facilita la creación de objetos según la información disponible.



la sobrecarga es posible porque mantiene el mismo nombre de la clase pero difiere en cantidad de parámetros recibidos no? dame el mismo ejemplo en java. En python y javascript será diferente verdad?

Exactamente. Esa es la idea principal.

En **C#** (al igual que en **Java**), la sobrecarga de constructores es posible porque:

- Todos los constructores tienen el **mismo nombre que la clase**.

- Se diferencian por su **firma**, es decir:
 - Cantidad de parámetros.
 - Tipo de parámetros.
 - Orden de los parámetros (si los tipos son distintos).

No es suficiente con cambiar únicamente el nombre de los parámetros; lo que debe cambiar es la **firma**.

Por ejemplo, estos dos constructores **sí** están sobrecargados:

Java

```
public Alumno(String nombre, String apellido)

public Alumno(String nombre, String apellido, int legajo, double notaFinal)
```

En cambio, estos **no** son válidos porque poseen exactamente la misma firma:

Java

```
public Alumno(String nombre, String apellido)

public Alumno(String n, String a)    // ERROR
```

Los nombres `nombre` o `n` no forman parte de la firma del método.

Ejemplo en Java

Clase Alumno.java

Java

```
public class Alumno {

    // Atributos
    private String nombre;
    private String apellido;
    private int legajo;
    private double notaFinal;

    // Constructor 1
    public Alumno(String nombre, String apellido) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.legajo = 0;
        this.notaFinal = 0;
    }

    // Constructor 2 (Sobrecargado)
    public Alumno(String nombre, String apellido,
                  int legajo, double notaFinal) {

        this.nombre = nombre;
        this.apellido = apellido;
        this.legajo = legajo;
    }
}
```

```
        this.notaFinal = notaFinal;
    }

    public void mostrarDatos() {
        System.out.println("Nombre: " + nombre);
        System.out.println("Apellido: " + apellido);
        System.out.println("Legajo: " + legajo);
        System.out.println("Nota Final: " + notaFinal);
        System.out.println("-----");
    }
}
```

App.java

```
</> Java
public class App {

    public static void main(String[] args) {

        Alumno alumno1 = new Alumno("Juan", "Pérez");

        Alumno alumno2 = new Alumno(
            "Ana",
            "Gómez",
            1254,
            8.75);

        alumno1.mostrarDatos();
        alumno2.mostrarDatos();
    }
}
```

Salida

```
Nombre: Juan
Apellido: Pérez
Legajo: 0
Nota Final: 0.0
-----
Nombre: Ana
Apellido: Gómez
Legajo: 1254
Nota Final: 8.75
-----
```

¿Qué ocurre en Python?

Aquí cambia el paradigma.

Python no admite sobrecarga de constructores.

Una clase solo puede tener un método `__init__()`.

Si escribimos dos, el segundo reemplaza al primero.

</> Python

```
class Alumno:

    def __init__(self, nombre):
        ...

    def __init__(self, nombre, apellido):
        ...
```

El primer constructor desaparece.

Por eso, en Python se utilizan otras estrategias, como:

- parámetros opcionales;
- valores por defecto;
- `*args`;
- `**kwargs`.

Ejemplo:

</> Python

```
class Alumno:

    def __init__(self, nombre, apellido,
                  legajo=0, nota_final=0):

        self.nombre = nombre
        self.apellido = apellido
        self.legajo = legajo
        self.nota_final = nota_final
```

Ahora puede crearse el objeto de ambas formas:

</> Python

```
a1 = Alumno("Juan", "Pérez")

a2 = Alumno("Ana", "Gómez", 1254, 8.75)
```

Aunque el efecto es similar al de la sobrecarga, técnicamente **no existe sobrecarga de constructores**: es un único constructor con parámetros opcionales.

¿Y en JavaScript?

Sucede algo muy parecido.

Una clase solo puede tener un único `constructor()`.

Esto **no** es válido:

```
</> JavaScript
class Alumno {

    constructor(nombre) { }

    constructor(nombre, apellido) { } // Error
}
```

Solo puede existir un constructor.

La solución habitual es utilizar parámetros opcionales:

```
</> JavaScript
class Alumno {

    constructor(nombre, apellido,
                legajo = 0,
                notaFinal = 0) {

        this.nombre = nombre;
        this.apellido = apellido;
        this.legajo = legajo;
        this.notaFinal = notaFinal;
    }
}
```

Y luego crear objetos de distintas formas:

```
</> JavaScript
const a1 = new Alumno("Juan", "Pérez");

const a2 = new Alumno("Ana", "Gómez", 1254, 8.75);
```

Nuevamente, el resultado es similar al de C# o Java, pero **no hay sobrecarga**: existe un único constructor con parámetros opcionales.

Comparación

Lenguaje	¿Permite sobrecarga de constructores?	¿Cómo se logra?
C#	✅ Sí	Mismo nombre, distinta firma (cantidad, tipo u orden de parámetros).
Java	✅ Sí	Igual que C#.
Python	❌ No	Un único <code>__init__()</code> con parámetros opcionales, <code>*args</code> o

Lenguaje	¿Permite sobrecarga de constructores?	¿Cómo se logra?
		<code>**kwargs</code> .
JavaScript	 No	Un único <code>constructor()</code> con parámetros opcionales o un objeto de configuración.

Esta comparación es muy útil para enseñar Programación Orientada a Objetos: permite mostrar que el concepto de **sobrecarga** forma parte del lenguaje en C# y Java, mientras que Python y JavaScript alcanzan una flexibilidad similar mediante parámetros opcionales y otras características del lenguaje, pero no implementan sobrecarga de constructores en sentido estricto.

